
Manual de preparación del entorno ESP32-C3

ESP-IDF, VS Code y USB Serial/JTAG

Entorno reproducible fijado en **ESP-IDF v6.0.1**

Taller de hardware hacking

Talk to Firmware – Introducción a Ingeniería Inversa en Hardware con JTAG

Estefanny Villalta – Fabio Ruiz

Índice

1	Información para participantes	1
1.1	Equipo que debe llevarse	1
1.2	Software que debe estar instalado	1
1.3	Configuración obligatoria de GNU/Linux	2
1.4	Comprobación previa y envío de evidencia	2
1.5	Conocimientos previos	3
2	Requisitos previos	4
2.1	Software que debe estar disponible	4
3	Actualización y dependencias de Linux	5
3.1	Debian, Ubuntu, Kali Linux y Linux Mint	5
3.2	Arch Linux y derivados	5
4	Comprobación de Visual Studio Code	6
4.1	Paquete requerido	6
4.2	Identificar la versión y el origen	6
5	Extensión oficial ESP-IDF	8
6	Abrir EIM desde la extensión	9
7	Instalación de ESP-IDF v6.0.1	10
7.1	Comprobación breve	11
8	Activación y verificación del entorno	12
8.1	Activar ESP-IDF en una terminal	12
8.2	Comprobar herramientas	12
8.3	Seleccionar la instalación en VS Code	13
9	Permisos USB/JTAG antes del taller	15
9.1	Instalar la regla oficial de OpenOCD	15
9.2	Agregar el usuario a los grupos necesarios	15
10	Preparación que puede completarse sin hardware	16
10.1	Crear y compilar el proyecto de prueba	16
10.2	Dejar el proyecto abierto en VS Code	17
10.3	Verificar OpenOCD sin conectarlo a una placa	17
10.4	Esquema de conexión de la placa	18
	Fuentes oficiales consultadas	19

Información para participantes

Este capítulo reúne en una sola lista los requisitos que deben cumplirse antes de asistir al taller. Su propósito es facilitar la revisión final del equipo, el software y la configuración del entorno.

1.1 Equipo que debe llevarse

Laptop y accesorios

Cada participante deberá llevar su **laptop y el cargador correspondiente**. El equipo deberá cumplir las siguientes condiciones:

- Procesador Intel o AMD con arquitectura **x86_64**. Se evitan equipos con arquitectura ARM debido a los problemas de compatibilidad que podrían presentarse.
- Al menos **8 GB de memoria RAM**.
- Un mínimo recomendado de **20 GB de almacenamiento libre** para descargar e instalar los archivos y programas requeridos.
- GNU/Linux de 64 bits instalado directamente en la computadora. Se admiten Debian y sus derivados, como Ubuntu, Kali Linux o Linux Mint; así como Arch Linux y sus derivados, como CachyOS, Manjaro o EndeavourOS.
- Al menos un puerto USB-A con capacidad para transferencia de datos. Si la laptop no dispone de este puerto, el participante deberá llevar el adaptador correspondiente.

No se utilizarán máquinas virtuales

El sistema GNU/Linux deberá estar instalado directamente en el equipo. Las máquinas virtuales pueden generar conflictos con el acceso a puertos seriales, dispositivos USB y permisos necesarios para JTAG.

1.2 Software que debe estar instalado

Antes de asistir al taller, cada participante deberá tener instalado:

- **Visual Studio Code**, mediante el paquete **.deb** en Debian y derivados o mediante el paquete nativo en Arch Linux y sus derivados. Deben evitarse las instalaciones Snap y Flatpak.
- La extensión oficial **ESP-IDF** para Visual Studio Code, publicada por **Espressif Systems**.
- **ESP-IDF Installation Manager (EIM)**.

- **Python 3.13.**
- **Git.**
- Las dependencias y herramientas básicas de desarrollo requeridas por ESP-IDF.
- La biblioteca `libusb`, necesaria para la comunicación con la interfaz USB-JTAG.

Herramientas administradas mediante EIM

La instalación de ESP-IDF deberá incluir y administrar los siguientes componentes:

- ESP-IDF v6.0.1.
- Toolchain RISC-V para el ESP32-C3.
- GDB para RISC-V.
- OpenOCD de Espressif.
- CMake y Ninja.
- Entorno virtual y paquetes de Python requeridos por ESP-IDF.
- `esptool` y las demás herramientas auxiliares incluidas con ESP-IDF.

1.3 Configuración obligatoria de GNU/Linux

Permisos que deben prepararse antes del taller

El sistema deberá contar con:

- Las reglas `udev` suministradas con OpenOCD.
- Los permisos de acceso a dispositivos USB y puertos seriales.

Estas configuraciones deben completarse siguiendo el Capítulo 9 antes de conectar el hardware del taller.

1.4 Comprobación previa y envío de evidencia

Plazo de entrega

A más tardar **48 horas antes de la actividad**, cada participante deberá enviar una captura de pantalla que demuestre que cuenta con el software requerido. La evidencia se enviará al siguiente correo:

josefabioruiz.uni@gmail.com

Para generar la evidencia, realice los siguientes pasos:

Paso 1. Active el entorno de ESP-IDF:

```
source ~/.espressif/v6.0.1/esp-idf/export.sh
```

Paso 2. Ejecute el siguiente bloque en la misma terminal:

```
code --version
python3 --version
git --version
idf.py --version
openocd --version
cmake --version
ninja --version
riscv32-esp-elf-gcc --version
riscv32-esp-elf-gdb --version
eim --version
```

Paso 3. Tome una captura en la que puedan leerse los comandos y sus respectivas versiones. Antes de enviarla, compruebe que no muestre contraseñas, tokens ni otra información sensible.

Criterio de aceptación

La captura será válida si todos los comandos muestran su versión y ninguno termina en `command not found`. En particular, `idf.py --version` deberá indicar **ESP-IDF v6.0.1**.

1.5 Conocimientos previos

Preparación recomendada

Cada participante deberá contar con conocimientos básicos sobre:

- Uso de la terminal, navegación entre directorios y ejecución de comandos.
- Programación básica en C.
- Conceptos elementales de compilación, flasheo y monitor serial.

No se requiere experiencia previa trabajando con hardware. Basta con tener curiosidad, disposición para seguir las instrucciones y ganas de aprender.

Requisitos previos

2.1 Software que debe estar disponible

El manual supone que el participante sabe administrar su distribución de Linux. No se explica cómo instalar herramientas generales. Si falta alguna, debe instalarse previamente siguiendo la documentación oficial correspondiente.

Componente	Versión o criterio	Comprobación
GNU/Linux	Sistema de 64 bits instalado directamente en una laptop x86-64.	<code>uname -m</code> debe mostrar <code>x86_64</code> .
Python 3	Versión 3.13, con <code>pip</code> , entornos virtuales y soporte SSL.	<code>python3 --version</code> , <code>python3 -m pip --version</code> y <code>python3 -m venv --help</code> .
Git	Versión mantenida por la distribución; no se exige un número exacto.	<code>git --version</code> .
Visual Studio Code	Edición estable instalada mediante paquete nativo. El equipo de referencia usó 1.125.0.	<code>code --version</code> .
Acceso administrativo	Usuario personal con permiso para ejecutar <code>sudo</code> ; no se trabajará iniciando sesión como <code>root</code> .	<code>sudo -v</code> .
Recursos	Conexión estable a Internet y 20 GB libres recomendados.	<code>df -h ~</code> .

Cuadro 2.1: Requisitos que deben cumplirse antes de instalar ESP-IDF.

Documentación oficial:

- Python: <https://www.python.org/downloads/>
- Git para Linux: <https://git-scm.com/download/linux>
- Visual Studio Code para Linux: <https://code.visualstudio.com/docs/setup/linux>
- Requisitos de EIM y ESP-IDF: <https://docs.espressif.com/projects/idf-im-ui/en/latest/prerequisites.html>

Actualización y dependencias de Linux

Ejecute **solamente** el bloque correspondiente a su distribución. Estos comandos actualizan el sistema e instalan bibliotecas requeridas por EIM, ESP-IDF, USB y JTAG. No instalan Python, Git ni VS Code, porque forman parte de los requisitos previos.

3.1 Debian, Ubuntu, Kali Linux y Linux Mint

```
sudo apt update
sudo apt upgrade -y

sudo apt install -y \
  wget flex bison gperf ccache build-essential \
  libffi-dev libssl-dev dfu-util libusb-1.0-0 \
  libgcrypt20 libglib2.0-0 libpixmap-1-0 \
  libsdl2-2.0-0 libslirp0 udev usbutils
```

3.2 Arch Linux y derivados

Arch no admite actualizaciones parciales. Sincronice primero todo el sistema:

```
sudo pacman -Syu

sudo pacman -S --needed \
  base-devel wget flex bison gperf ccache \
  libffi openssl dfu-util libusb usbutils \
  libgcrypt glib2 pixmap sdl2 libslirp \
  webkit2gtk-4.1
```

Comprobación

Si el gestor de paquetes informa que una dependencia ya está instalada, no es un error. Si indica que un nombre no existe, consulte la lista actualizada para su distribución en la documentación de requisitos de EIM antes de continuar.

Comprobación de Visual Studio Code

4.1 Paquete requerido

Para evitar problemas de aislamiento, rutas y acceso a herramientas externas, el taller utiliza:

- en Debian y derivados, el paquete oficial `.deb` de Microsoft llamado `code`;
- en Arch y derivados, el paquete nativo `visual-studio-code-bin`.

No se utilizarán instalaciones Snap ni Flatpak. Si VS Code no está instalado o usa uno de esos formatos, sustitúyalo antes de continuar. La guía oficial está en <https://code.visualstudio.com/docs/setup/linux>.

4.2 Identificar la versión y el origen

```
command -v code
readlink -f "${command -v code}"
code --version | head -n 1
```

En Debian y derivados, confirme el paquete:

```
dpkg-query -W code
```

En Arch y derivados:

```
pacman -Q visual-studio-code-bin
```

Resultado esperado

`command -v code` debe resolver normalmente a `/usr/bin/code`. La ruta no debe apuntar a `/snap/bin`. El gestor de paquetes debe mostrar `code` o `visual-studio-code-bin`, según la distribución.

```
fabio@DELL:~$ command -v code  
readlink -f "$(command -v code)"  
code --version | head -n 1  
/usr/bin/code  
/usr/share/code/bin/code  
1.135.0  
fabio@DELL:~$ dpkg-query -W code  
code      1.135.0-1787669172  
fabio@DELL:~$
```

Figura 4.1: Terminal con la versión de VS Code y la comprobación de su paquete nativo.

Extensión oficial ESP-IDF

1. Abra Visual Studio Code sin usar `sudo`.
2. Presione `Ctrl+Shift+X` para abrir *Extensions*.
3. Busque ESP-IDF.
4. Confirme que el publicador sea **Espressif Systems**.
5. Seleccione *Install*.

Compruebe la instalación y la versión registrada:

```
code --list-extensions --show-versions \
| grep '^espressif\.esp-idf-extension@'
```

Resultado esperado

Debe aparecer una sola línea que comience con `espressif.esp-idf-extension@`.

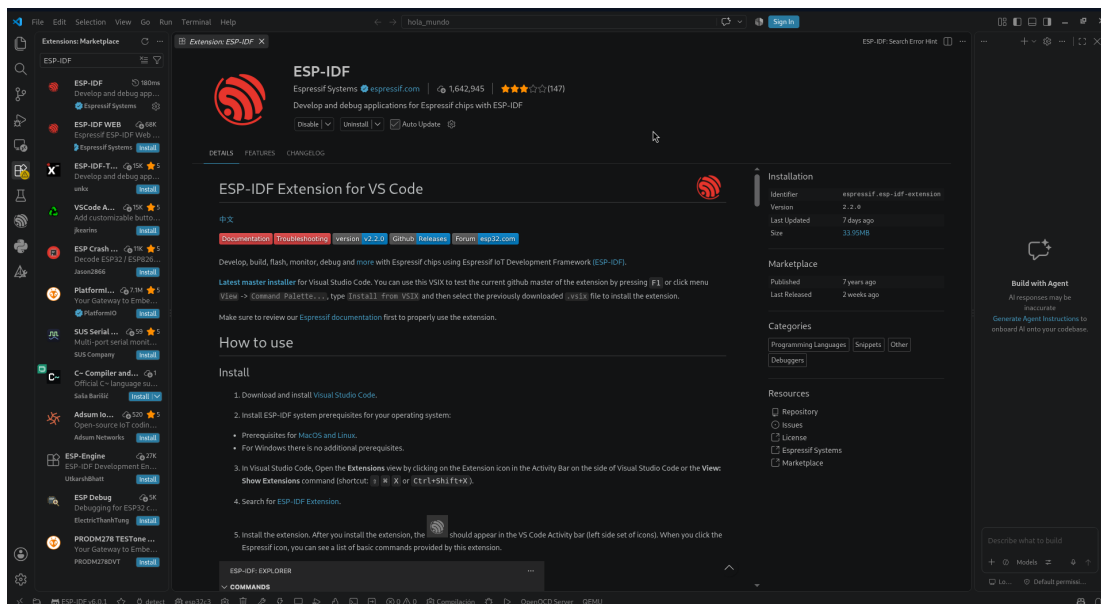


Figura 5.1: Extensión ESP-IDF instalada y publicador Espressif Systems visible.

Abrir EIM desde la extensión

EIM es el administrador que descargará ESP-IDF y sus herramientas. Para este taller se abrirá **únicamente desde la extensión de VS Code**; no es necesario instalarlo mediante comandos.

1. Abra la paleta de comandos con F1.
2. Escriba ESP-IDF: Open ESP-IDF Installation Manager.
3. Seleccione el comando y espere la notificación de descarga.
4. Cuando aparezca la ventana de EIM, continúe con el capítulo siguiente.

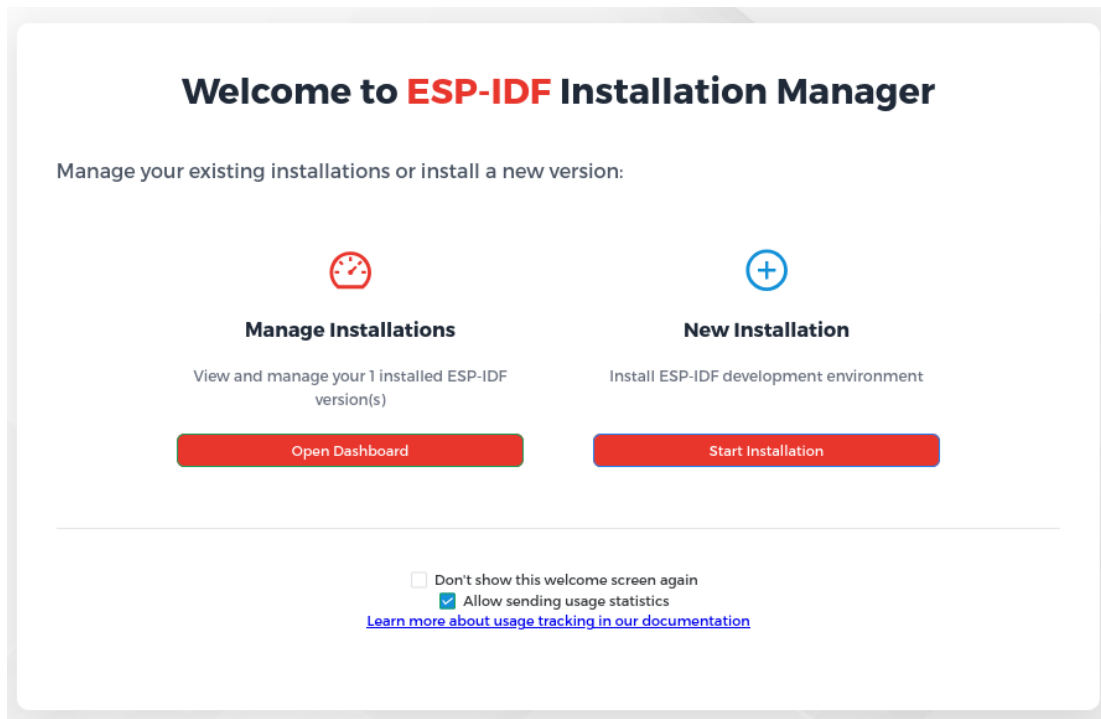


Figura 6.1: Ventana de Open ESP-IDF Installation Manager.

Instalación de ESP-IDF v6.0.1

1. En EIM, abra *New Installation* y seleccione *Start Installation*.
2. Elija **Expert/Custom Installation**. La opción rápida puede seleccionar una revisión estable diferente.
3. Mantenga como directorio base `~/.espressif`.
4. Seleccione explícitamente **ESP-IDF v6.0.1**.
5. Mantenga las herramientas obligatorias para ESP32-C3. EIM instalará el compilador y GDB RISC-V, OpenOCD, CMake, Ninja, `esptool` y el entorno Python de ESP-IDF.
6. Inicie la instalación y espere el mensaje de finalización. No suspenda el equipo ni cambie de red durante la descarga.

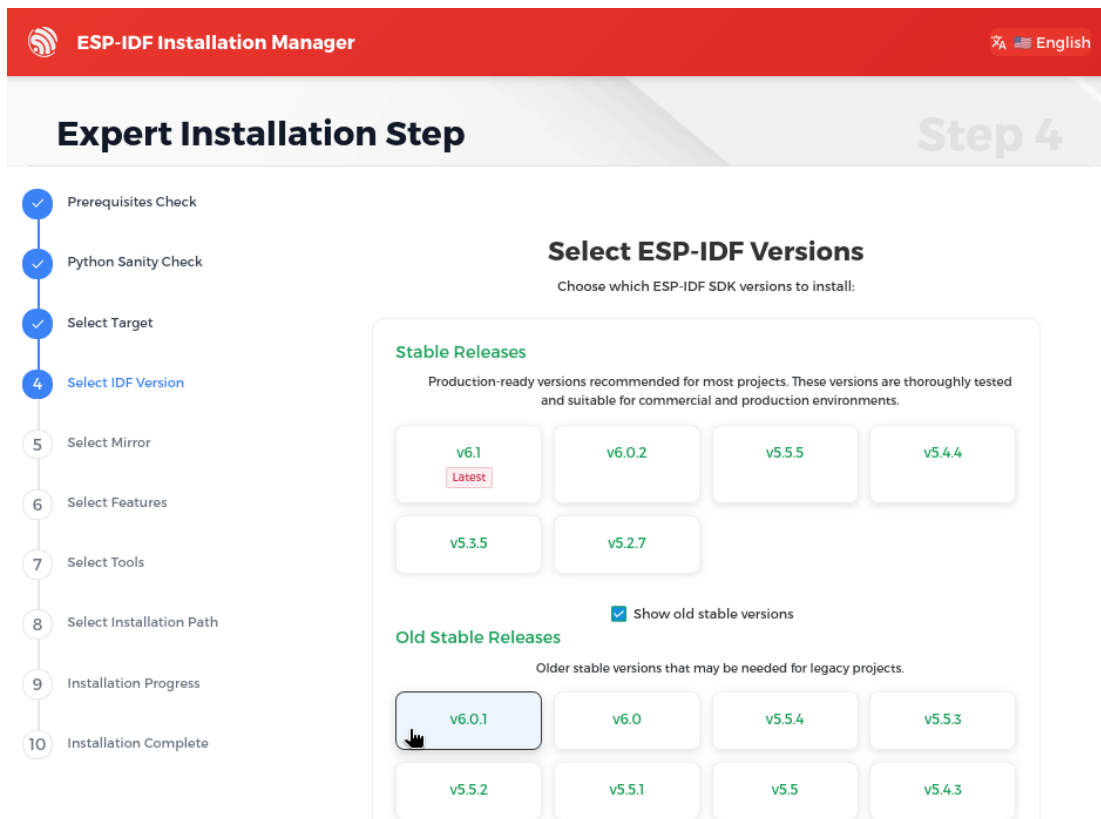


Figura 7.1: EIM con ESP-IDF v6.0.1 seleccionado en la instalación personalizada.

7.1 Comprobación breve

```
test -f ~/.espressif/tools/eim_idf.json \  
  && echo "Registro de EIM: OK"  
  
test -d ~/.espressif/v6.0.1/esp-idf \  
  && echo "ESP-IDF v6.0.1: OK"
```

Ambas líneas deben finalizar con OK. La ruta puede verse también en la lista de instalaciones administradas por EIM.

Activación y verificación del entorno

8.1 Activar ESP-IDF en una terminal

Cada terminal nueva debe cargar el entorno:

```
source ~/.espressif/v6.0.1/esp-idf/export.sh
```

source es necesario

No ejecute el archivo como un programa separado. `source` permite que el script configure las variables de la terminal actual.

8.2 Comprobar herramientas

```
python3 --version
git --version
idf.py --version
openocd --version
cmake --version
ninja --version
riscv32-esp-elf-gcc --version
riscv32-esp-elf-gdb --version
eim --version
```

Resultado esperado

Ningún comando debe terminar en `command not found`. Lo indispensable es que `idf.py --version` muestre ESP-IDF v6.0.1.

```

fabio@DELL:~$ source ~/.espressif/v6.0.1/esp-idf/export.sh
Checking "python3" ...
Python 3.13.5
"python3" has been detected
Activating ESP-IDF 6.0
Setting IDF PATH to '/home/fabio/.espressif/v6.0.1/esp-idf'.
* Checking python version ... 3.13.5
* Checking python dependencies ... OK
* Deactivating the current ESP-IDF environment (if any) ... OK
* Establishing a new ESP-IDF environment ... OK
* Identifying shell ... bash
* Detecting outdated tools in system ... Found tools that are not used by active ESP-IDF version.
For removing old versions of components, python use command 'python /home/fabio/.espressif/v6.0.1/esp-idf/tools/idf_tools.py uninstall'
To free up even more space, remove installation packages of those tools.
Use option python /home/fabio/.espressif/v6.0.1/esp-idf/tools/idf_tools.py uninstall --remove-archives.
* Shell completion ... Autocompletion code generated

Done! You can now compile ESP-IDF projects.
Go to the project directory and run:

idf.py build
fabio@DELL:~$ python3 --version
python3 3.13.5
git --version
git version 2.47.3
idf.py --version
idf.py 6.0.1
openocd --version
openocd 0.12.1
cmake --version
cmake 4.0.3
ninja --version
ninja 1.12.1
riscv32-esp-elf-gcc --version
riscv32-esp-elf-gcc (crosstool-NG esp-15.2.0_20251204) 15.2.0
Python 3.13.5
git version 2.47.3
ESP-IDF v6.0.1
Open On-Chip Debugger v0.12.0-esp32-20260304 (2026-03-04-20:07)
Licensed under GNU GPL v2
For bug reports, read
    http://openocd.org/doc/doxygen/bugs.html
cmake version 4.0.3

CMake suite maintained and supported by Kitware (kitware.com/cmake).
1.12.1
riscv32-esp-elf-gcc (crosstool-NG esp-15.2.0_20251204) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

GNU gdb (esp-gdb) 16.3.20250913
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
fabio@DELL:~$

```

Figura 8.1: Versiones de ESP-IDF, OpenOCD, CMake, Ninja, GCC y GDB con el entorno activo.

8.3 Seleccionar la instalación en VS Code

1. Abra VS Code y presione F1.
2. Ejecute ESP-IDF: Select Current ESP-IDF Version.
3. Seleccione la entrada correspondiente a v6.0.1.
4. Ejecute ESP-IDF: Doctor Command.

Las instalaciones de EIM se detectan mediante `~/.espressif/tools/eim_idf.json`. El informe de *Doctor* debe reconocer la versión y las rutas de sus herramientas.

```

1 ----- ESP-IDF Extension for Visual Studio Code report -----
2 OS linux x64 6.12.100+deb13-amd64
3 System environment variable IDF_PYTHON_ENV_PATH
4 undefined
5 System environment variable PATH
6 <HOMEPATH>/local/bin:<HOMEPATH>/local/bin:<HOMEPATH>/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games
7 System environment variable PYTHON
8 undefined
9 Visual Studio Code Remote name undefined
10 Visual Studio Code version 1.135.0
11 Visual Studio Code language en
12 Visual Studio Code shell /usr/bin/bash
13 Visual Studio Code app name Visual Studio Code
14 ESP-IDF Extension version 2.2.0
15 Workspace folder <HOMEPATH>/Documentos/prueba
16 ----- Extension configuration settings -----
17 ESP-ADF Path (idf.customExtraVars["ADF_PATH"]) undefined
18 ESP-IDF Path (Project setup IDF_PATH) <HOMEPATH>/Documentos/taller/.espressif/v6.0.1/esp-idf
19 Custom extra paths <HOMEPATH>/espressif/tools/cmake/4.0.3/bin:<HOMEPATH>/espressif/tools/esp-clang/esp-20.1.1_20250829/esp-clang/bin:<HOMEPATH>/espressif/tools/esp-clang/esp-20.1.1_20250829/esp-clang/bin:<HOMEPATH>/espressif/eim_gui/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/local/anaconda3/bin
20 ESP-IDF Project Setup Variables
21 PATH: <HOMEPATH>/espressif/tools/cmake/4.0.3/bin:<HOMEPATH>/espressif/tools/esp-clang/esp-20.1.1_20250829/esp-clang/bin:<HOMEPATH>/espressif/tools/esp-clang/esp-20.1.1_20250829/esp-clang/bin:<HOMEPATH>/espressif/eim_gui/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/local/anaconda3/bin
22 SYSTEM PATH: <HOMEPATH>/local/bin:<HOMEPATH>/local/bin:<HOMEPATH>/local/bin:<HOMEPATH>/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/local/anaconda3/bin
23 ESP_IDF_VERSION: 6.0
24 IDF_TOOLS_PATH: <HOMEPATH>/espressif/tools
25 IDF_COMPONENT_LOCAL_STORAGE_URL: file://<HOMEPATH>/espressif/tools
26 IDF_PATH: <HOMEPATH>/Documentos/taller/.espressif/v6.0.1/esp-idf
27 ESP_ROM_ELF_DIR: <HOMEPATH>/espressif/tools/esp-rom-elfs/20241011/
28 OPENOCD_SCRIPTS: <HOMEPATH>/espressif/tools/openocd-esp32/v0.12.0-esp32-20260304/openocd-esp32/share/openocd/scripts
29 IDF_PYTHON_ENV_PATH: <HOMEPATH>/espressif/tools/python/v6.0.1/venv
30 ESP_CLANG_LIBS_PATH: <HOMEPATH>/espressif/tools/esp-clang-libs/esp-20.1.1_20250829/esp-clang/lib
31 PYTHON: <HOMEPATH>/espressif/tools/python/v6.0.1/venv/bin/python
32 User extra vars (idf.customExtraVars)
33 Virtual environment Python path (computed) <HOMEPATH>/espressif/tools/python/v6.0.1/venv/bin/python3
34 Serial port (idf.port) detect
35 OpenOCD Configs (idf.openOcdConfigs) interface/ftdi/esp32_devkitj_v1.cfg,board/esp32-wrover.cfg
36 OpenOCD log level (idf.openOcdDebugLevel) 2
37 OpenOCD launch arguments (idf.openOcdLaunchArgs)
38 ESP-IDF Tools Path <HOMEPATH>/espressif/tools
39 Git Path (idf.gitPath) /usr/bin/git
40 Notification Mode (idf.notificationMode) All
41 Flash type (idf.flashType)
42 Flash partition to use (idf.flashPartitionToUse) all
43 ----- Configurations access -----
44 Access to ESP-ADF Path (idf.customExtraVars["ADF_PATH"]) false
45 Access to ESP-IDF Path (Project setup IDF_PATH) true
46 Access to ESP-IDF Custom extra paths
47 Access to <HOMEPATH>/espressif/tools/cmake/4.0.3/bin: true
48 Access to <HOMEPATH>/espressif/tools/esp-clang/esp-20.1.1_20250829/esp-clang/bin: true

```

Figura 8.2: ESP-IDF v6.0.1 seleccionado y resumen del Doctor Command en VS Code.

Permisos USB/JTAG antes del taller

Este capítulo prepara los permisos, pero **no puede probarlos todavía** porque la placa y el adaptador se entregarán durante el taller.

9.1 Instalar la regla oficial de OpenOCD

Active el entorno y compruebe que la regla existe:

```
source ~/.espressif/v6.0.1/esp-idf/export.sh  
ls "$OPENOCD_SCRIPTS/../../contrib/60-openocd.rules"
```

Si el archivo aparece, cópielo y recargue udev:

```
sudo cp "$OPENOCD_SCRIPTS/../../contrib/60-openocd.rules" \  
    /etc/udev/rules.d/60-openocd.rules  
  
sudo udevadm control --reload-rules  
sudo udevadm trigger
```

9.2 Agregar el usuario a los grupos necesarios

En Debian y derivados:

```
getent group plugdev || sudo groupadd --system plugdev  
sudo usermod -aG plugdev,dialout "$USER"
```

En Arch y derivados:

```
sudo usermod -aG uucp "$USER"
```

Cierre completamente la sesión gráfica, vuelva a entrar y ejecute:

```
id
```

Preparación que puede completarse sin hardware

10.1 Crear y compilar el proyecto de prueba

Abra una terminal, active ESP-IDF y copie el ejemplo oficial:

```
source ~/.espressif/v6.0.1/esp-idf/export.sh

mkdir -p ~/esp
cd ~/esp
cp -r "$IDF_PATH/examples/get-started/hello_world" hola_mundo
cd hola_mundo

idf.py set-target esp32c3
idf.py build
```

Resultado esperado

La compilación debe terminar con un mensaje equivalente a `Project build complete`. La carpeta `build` debe contener archivos `.elf`, `.bin` y `flasher_args.json`.

```
ls -lh build/*.elf build/*.bin build/flasher_args.json
```

```
Project build complete. To flash, run:
idf.py flash
or
idf.py -p PORT flash
or
python -m esptool --chip esp32c3 -b 460800 --before default-reset --after hard-reset write-flash --flash-mode dio --flash-size 2MB --flash-freq 80m 0x0 build/bootloader/bootloader.bin 0x8000 build/partition-table/partition-table.bin 0x10000 build/hello_world.bin
or from the "/home/fabio/esp/hola_mundo/build" directory
python -m esptool --chip esp32c3 -b 460800 --before default-reset --after hard-reset write-flash "@flash_args"
fabio@DELL:~/esp/hola_mundo$ ls -lh build/*.elf build/*.bin build/flasher_args.json
-rw-rw-r-- 1 fabio fabio 913 ago 30 18:23 build/flasher_args.json
-rw-rw-r-- 1 fabio fabio 119K ago 30 18:23 build/hello_world.bin
-rwxrwxr-x 1 fabio fabio 3,0M ago 30 18:23 build/hello_world.elf
fabio@DELL:~/esp/hola_mundo$
```

Figura 10.1: Compilación correcta de `hola_mundo` para el objetivo `esp32c3`.

10.2 Dejar el proyecto abierto en VS Code

```
cd ~/esp/hola_mundo  
code .
```

En VS Code:

1. confirme ESP-IDF v6.0.1;
2. seleccione esp32c3 con ESP-IDF: Set Espressif Device Target; y
3. ejecute ESP-IDF: Build your Project.

No seleccione todavía un puerto, no flashee y no inicie depuración.

10.3 Verificar OpenOCD sin conectarlo a una placa

```
source ~/.espressif/v6.0.1/esp-idf/export.sh  
openocd --version  
  
test -f "$OPENOCD_SCRIPTS/board/esp32c3-builtin.cfg" \  
&& echo "Configuración OpenOCD: OK"
```

Preparación en casa completada

Antes del taller deben cumplirse únicamente estos puntos:

- ESP-IDF v6.0.1 y sus herramientas responden.
- VS Code reconoce la instalación y el proyecto.
- *Doctor Command* no informa herramientas ausentes.
- La regla udev está copiada y la sesión se reinició.
- `hola_mundo` compila para `esp32c3`.
- Existe `board/esp32c3-builtin.cfg`.

No es necesario demostrar USB, flasheo, monitor ni JTAG en casa.

10.4 Esquema de conexión de la placa

Realice el cableado con el adaptador desconectado de la laptop. Guíese por las etiquetas eléctricas, no solo por el color del cable.

Señal USB	Pin de la ESP32-C3-DevKitM-1	Color habitual
VBUS / +5V	5V	Rojo
GND	GND	Negro
D-	GPI018	Blanco
D+	GPI019	Verde

Cuadro 10.1: Conexión del USB nativo al controlador USB Serial/JTAG.

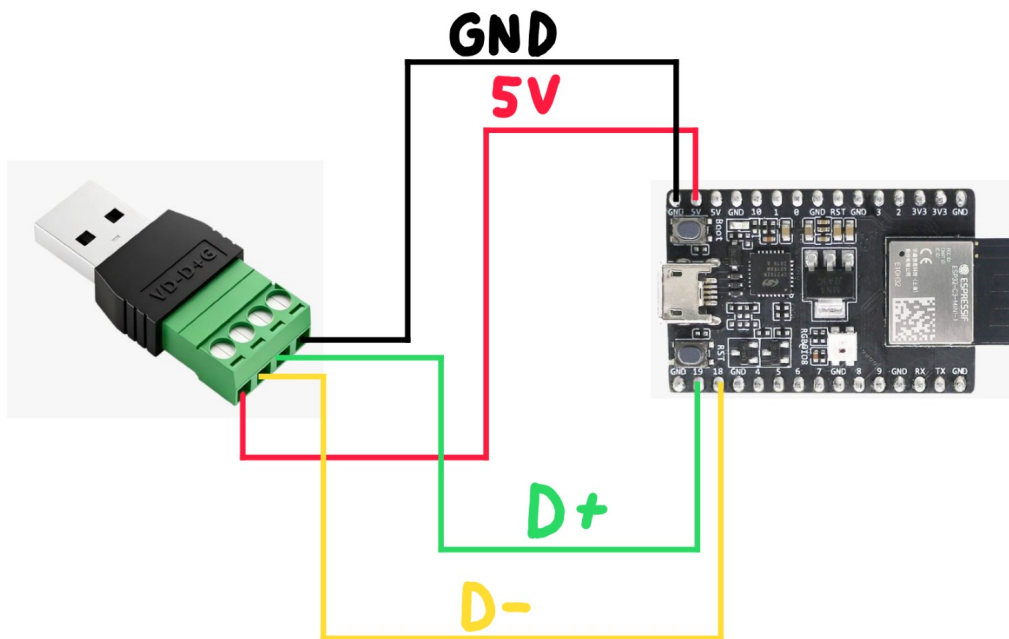


Figura 10.2: Esquema de la conexión correcta de los jumpers a la ESP32C3, debe guiarse por la ubicación de los cables y las imágenes, ya que los colores de los cables pueden variar

Si algo falla

Si realiza la conexión y las luces de la ESP32C3 encienden pero el JTAG no es detectado mediante el lsusb, entonces pruebe invertir los cables de enmedio D- y D+. Es muy importante que no invierta los cables de alimentación 5V y GND ni que tampoco los coloque en otro pin que no haya sido el indicado, ya que esa acción incorrecta puede quemar la ESP32C3 y dejarla inservible.

Fuentes oficiales consultadas

- [1] Espressif Systems, *ESP-IDF Installation Manager: Prerequisites*. <https://docs.espressif.com/projects/idf-im-ui/en/latest/prerequisites.html>
- [2] Espressif Systems, *ESP-IDF Installation Manager*. <https://docs.espressif.com/projects/idf-im-ui/en/latest/>
- [3] Espressif Systems, *Install ESP-IDF and Tools – ESP-IDF Extension for VS Code*. <https://docs.espressif.com/projects/vscode-esp-idf-extension/en/latest/installation.html>
- [4] Microsoft, *Installing Visual Studio Code on Linux*. <https://code.visualstudio.com/docs/setup/linux>
- [5] Espressif Systems, *ESP-IDF Programming Guide v6.0.1 – ESP32-C3*. <https://docs.espressif.com/projects/esp-idf/en/v6.0.1/esp32c3/>
- [6] Espressif Systems, *USB Serial/JTAG Controller Console – ESP32-C3, ESP-IDF v6.0.1*. <https://docs.espressif.com/projects/esp-idf/en/v6.0.1/esp32c3/api-guides/usb-serial-jtag-console.html>
- [7] Espressif Systems, *Configure ESP32-C3 Built-in JTAG Interface*. <https://docs.espressif.com/projects/esp-idf/en/stable/esp32c3/api-guides/jtag-debugging/configure-built-in-jtag.html>
- [8] Espressif Systems, *JTAG Debugging – ESP32-C3*. <https://docs.espressif.com/projects/esp-idf/en/stable/esp32c3/api-guides/jtag-debugging/index.html>